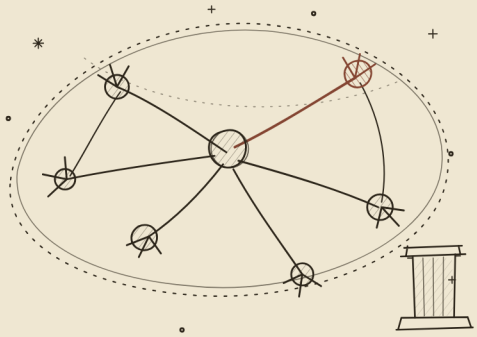
EXPEDIENTE · GUÍA DEL USUARIO

Proyecto Alejandría

Tu segundo cerebro



cada concepto, una neurona; cada relación, una sinapsis

1. Qué es y para qué sirve

Proyecto Alejandría es una aplicación de escritorio para Windows que convierte lo que aprendes en un **universo en 3D**. Cada concepto (la derivada, la distribución de Poisson, el origen histórico de la probabilidad, una teoría económica…) es una **neurona**: guarda su teoría, un glosario y referencias; y, si el concepto es matemático, ejercicios resueltos y un **laboratorio** donde puedes mover parámetros y ver cómo cambia el modelo.

Las neuronas se conectan entre sí mediante **sinapsis**. Las propone una inteligencia artificial que corre **en tu propio ordenador** (Ollama), explica por qué cree que dos conceptos se relacionan y te deja aceptar o rechazar cada una. Nada sale de tu equipo: no hay cuentas ni envío de datos, y la nube solo entra si tú eliges una carpeta para sincronizar tus PCs.

Sirve para estudiar y repasar, para construir un mapa propio de una materia y para entender un modelo tocándolo, no solo leyéndolo. Es un proyecto personal de código abierto (licencia MIT) y por ahora solo funciona en Windows 10 y 11.

Pensado para que tu conocimiento sea tuyo: en una carpeta local, exportable y sin depender de nadie.

2. De un vistazo

Neuronas	Formales (matemáticas), históricas o conceptuales, y teóricas no matemáticas: cada tipo con su prompt.
Sinapsis	Conexiones propuestas por una IA local, con su razón; tú decides.
Laboratorios	Simulaciones en Python que se ejecutan dentro de la ventana, sin internet.
Editar y revisar	Corrige cualquier neurona a mano o pide a una IA externa que la revise; la papelera te deja recuperar lo destruido.
Agenda	Contactos privados y para qué los guardaste; solo en tu equipo.
Varios PCs	Una carpeta de tu nube (OneDrive, Dropbox…) mantiene tus neuronas iguales en dos ordenadores.
Actualizaciones	La app avisa de las versiones nuevas y se actualiza sola cuando tú lo pides.
Privacidad	Todo en local; el servidor solo escucha en tu equipo.

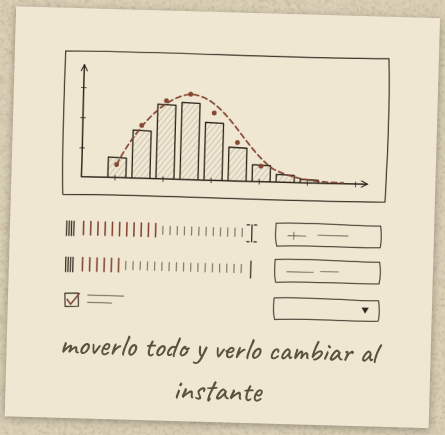
Cómo se usa

paso a paso

3. Funcionalidades y mecánicas

Crear una neurona

1. Pulsa `[ + Inyectar neurona ]` (abajo a la izquierda).
2. Elige el **tipo de concepto** en el desplegable (más abajo) y copia el prompt: *Fuentes*, si tienes tus apuntes o papers, o *Exploración libre*, que pide a la IA consultar también Google Scholar.
3. Pégalo en tu IA favorita con el tema que quieras y pega aquí el JSON que te devuelva (si le faltan comas o escapes, la app lo repara, y quita marcas de cita como `[reference:12]`).
4. Pulsa `[ Asimilar neurona ]`: se guarda, se clasifica en una rama y la IA local busca sus conexiones (verás el pilar girando y una barra de avance).



moverlo todo y verlo cambiar al instante

Tipos de neurona



fórmulas, historia o teoría

El desplegable del Sintetizador cambia el prompt: **Automático** (la IA decide el tipo; ante la duda, formal).

Formal. Matemático: teoría de doctorado, glosario, ejercicios y laboratorio (el prompt de siempre).

Histórico o conceptual. Personas, épocas, escuelas e ideas: relato, glosario y referencias; sin ejercicios ni laboratorio.

Teórico no matemático. Marcos conceptuales (economía, filosofía de la ciencia…): teoría argumentada y casos de análisis.

Explorar el universo

Arrastra para girar, rueda para acercarte y haz clic en una neurona para abrirla. Si te acercas mucho, los nombres cercanos aparecen tenues. La barra superior busca por nombre, rama o glosario (flechas y Enter para elegir); `[ Centrar vista ]` vuelve a la vista completa. Cada neurona **conserva su sitio** al recargar o añadir otras, y siempre queda una distancia mínima entre ellas.

El panel de una neurona

Seis pestañas: **Teoría**, **Glosario**, **Laboratorio**, **Ejercicios** (con la solución oculta hasta que la pidas), **Referencias** y **Sinapsis**. En las históricas o teóricas, Ejercicios o Laboratorio pueden estar vacíos (puedes crear un laboratorio a mano). Una insignia `[F]`, `[H]` o `[T]` indica su tipo. Desde la cabecera puedes editarla, pedir una revisión, corregir el clúster a mano o mandarla a la papelera.

Sinapsis

Cada conexión nueva queda pendiente con la razón que dio la IA (en castellano). También propone vínculos históricos o conceptuales (misma persona o escuela, influencia clara), como «relacionados». `[ Aceptar ]` la consolida; `[ Rechazar ]` la elimina y **no vuelve a proponerse**. También puedes crear una conexión manual con tu propio motivo. Si Ollama no está en marcha, la app lo arranca sola.

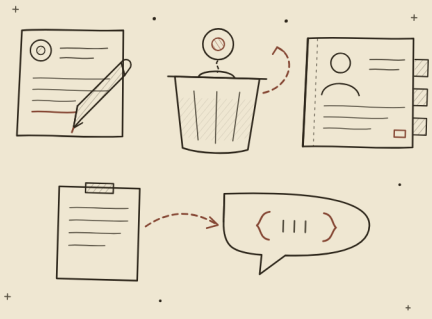
Laboratorios

Mueve las barras `[#####-----]` (con el ratón o las flechas del teclado) y el gráfico se redibuja al momento: barras, líneas, áreas, histogramas, imágenes de matrices, escala logarítmica… Con `[ Editar laboratorio ]` añades o quitas variables, cambias rangos y editas el código Python. Un laboratorio que viene de un JSON pegado pide tu aprobación antes de ejecutarse: léelo antes de aceptar.

Editar y varios PCs

desde la 0.4

4. Editar, revisar y recuperar



editar, pedir revisión, papelera y agenda

Editar. `[ Editar ]` abre el nombre, el tipo, la teoría (con vista previa de las fórmulas en vivo), el glosario, los ejercicios y las referencias. El clúster y el laboratorio no se tocan.

Pedir revisión. `[ Pedir revisión ]` copia un prompt con la neurona para que una IA externa la revise (primero las referencias: corregir o quitar, nunca inventar). Pega su respuesta en `[ Editar ]` → `[ Pegar JSON revisado ]`: la app la lee aunque venga con comas o comillas mal puestas y la aplica al formulario; tú decides si guardas.

Papelera. `[ A la papelera ]` ya no destruye: la neurona sale del universo pero queda 7 días en `[ Papelera ]` (barra superior), donde puedes restaurarla con sus conexiones o borrarla de verdad; pasado ese tiempo se borra sola. La papelera también se sincroniza entre tus PCs.

Agenda. `[ Agenda ]` guarda contactos privados: quién es, correo o teléfono, tipo (académico, corporativo u otro) y qué necesitas de esa persona en el futuro. Se busca, se filtra y se exporta aparte; **nunca** viaja con la sincronización ni con tus neuronas.

5. Varios ordenadores



una carpeta de tu nube los mantiene iguales

Si usas la app en dos PCs, en **Ajustes → Sincronización entre PCs** eliges en los dos una carpeta que ya sincronice tu nube (OneDrive, Dropbox, Google Drive, Syncthing…). Cada PC deja ahí lo suyo, en su propia subcarpeta, y lee lo de los demás: **sin servidor ni cuentas nuevas.**

- Viajan las **ediciones**, las **neuronas nuevas**, los **borrados** y qué conexiones aceptaste o rechazaste. No viajan los ajustes ni los cálculos internos de cada PC.
- Se sincroniza **al abrir la app** y cuando pulsas `[ Sincronizar ahora ]`; no hay nada corriendo todo el rato.
- Si editas lo mismo en dos PCs a la vez, gana lo más reciente y lo otro se guarda aparte, en la carpeta `conflictos`. Si la carpeta no está disponible no se cambia nada, y si de golpe desaparecen muchas neuronas locales se restauran en vez de borrarse.

6. Actualizaciones

Al abrirse, la app pregunta a github.com si hay una versión nueva (no envía nada tuyo; se puede apagar en Ajustes). Si la hay, aparece un **cartel pequeño** en la esquina de abajo a la derecha: la versión, un «qué hay de nuevo» plegado y dos botones, `[ Luego ]` y `[ Actualizar ]`.

`[ Luego ]` oculta el cartel hasta la próxima vez que abras la app; no se instala nada si no pulsas `[ Actualizar ]`. Al pulsarlo se descarga el instalador y la app comprueba su **firma digital** con la clave pública que lleva dentro: solo se instala lo firmado por el proyecto. Después se cierra, una ventanita mínima la actualiza y vuelve a abrirse con un aviso de 5 segundos. Tus neuronas no se tocan. Las versiones anteriores a la 0.2.0 se instalan a mano una vez.

Instalar y por dentro

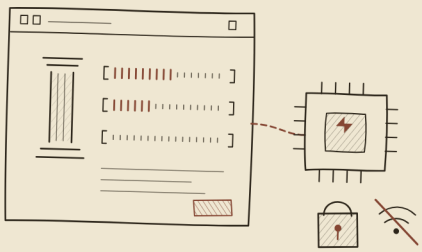
Windows 10 / 11

7. Ajustes

Activa o desactiva la IA local, elige el modelo, **exporta e importa** tus neuronas, carga tres de ejemplo, repite el tutorial, mira las licencias de terceros y controla las **actualizaciones** y la **sincronización**. Importar nunca pisa una neurona que ya tengas.

8. Instalación y requisitos

- **Sistema:** Windows 10 u 11 de 64 bits, sin ser administrador, con el *Visual C++ Redistributable* actual (14.40 o posterior); el instalador lo comprueba.
- **Descarga:** el instalador `Proyecto-Alejandria-Setup.exe` está en la página de releases del repositorio. Opcional: verifica el hash con `SHA256SUMS.txt`.
- **Espacio:** el «motor» (Python y librerías) ocupa unos 500 MB, el modelo de clasificación ~240 MB y Ollama, si lo eliges, 1,6 GB más el modelo.
- **Internet:** solo para instalar (se baja todo una vez), buscar actualizaciones y sincronizar por tu nube.



todo en tu equipo, sin internet al usarlo

Aviso de Windows. El instalador no está firmado (la firma de código cuesta dinero y es un proyecto personal), así que SmartScreen mostrará «Windows protegió su PC». Pulsa Más información y luego Ejecutar de todas formas, solo si lo has descargado de la página oficial del proyecto.

El asistente te deja elegir dónde van la aplicación, el motor y tus neuronas, y si instalas Ollama y con qué modelo. Con *Examinar* cada parte va a su subcarpeta (`Aplicacion`, `Motor`, `Neuronas`, `Modelos`), o marca «Instalarlo todo en una sola carpeta». Si algo falla, mira `instalacion.log` en la carpeta del motor y vuelve a ejecutarlo: retoma lo que falte.

9. Detalles técnicos

Interfaz	Tauri 2 (Rust) + React + Three.js; instalador NSIS propio.
Servidor local	FastAPI en `127.0.0.1`, datos en Parquet, embeddings con fastembed.
IA	Ollama (`qwen2.5:1.5b`, `phi3:mini` …) para juzgar sinapsis; opcional.
Laboratorios	Pyodide (Python en WebAssembly: numpy, scipy, matplotlib, pandas) aislado en un worker; CSP estricta.
Prompts	Cuatro modos (automático, formal, histórico, teórico) generados desde una plantilla común.
Actualizador	Plugin de actualización de Tauri con firma minisign (Ed25519).
Sincronización	Cada PC escribe solo en su subcarpeta; conflictos guardados aparte.

Ejecutarlo desde el código (Python 3.11+, Node.js 20+ y Rust)

```
alejandria.bat
cd frontend && npm install && npm run tauri dev
```

Código MIT en `github.com/MarioGreks/proyecto-alejandria`. Tests: `unittest` y `npm test`.